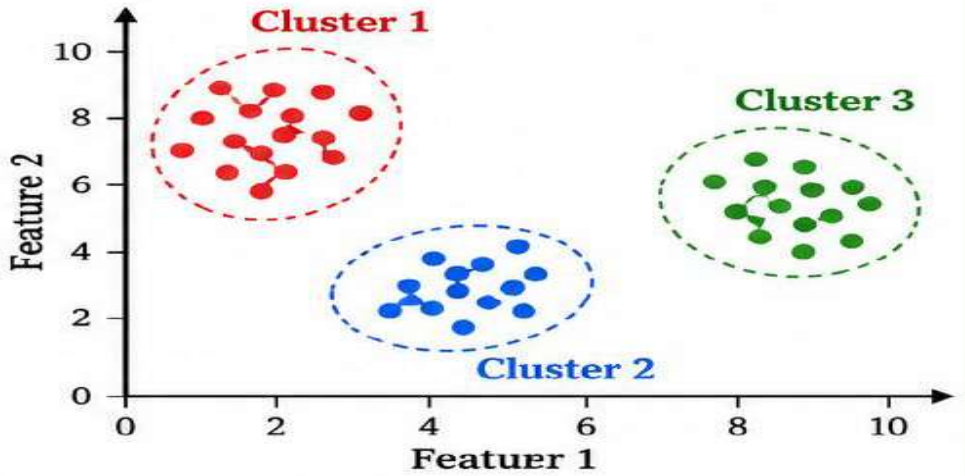
CLUSTERING

- **Clustering** is the task of dividing a population or data points into a number of groups (clusters) such that:
  - Data points in the same group are more **similar** to each other.
  - Data points in different groups are **dissimilar** to each other.
- It is a collection of objects based on the similarity and dissimilarity between them.
- Clustering is a type of **unsupervised learning** (no class labels are used).
- It helps in discovering inherent structure or patterns in data.

Example of Clustering



Types of Clustering

- Clustering is a type of **unsupervised learning** wherein data points are grouped into different sets based on their degree of similarity.
- The main types of clustering are:

Types of Clustering

Hierarchical Clustering

Builds a hierarchy (tree-like structure) of clusters by either merging smaller clusters or splitting larger clusters.

Agglomerative Clustering

- Bottom-up approach.
- Starts with each data point as a single cluster.
- Iteratively merges the closest clusters.
- Results in a dendrogram (tree structure).

Divisive Clustering

- Top-down approach.
- Starts with all data points in a single cluster.
- Iteratively splits clusters into smaller clusters.
- Also results in a dendrogram.

Partitioning Clustering

Divides the data into a predefined number of clusters by optimizing a criterion (e.g., minimizing within-cluster distance).

K-Means Clustering

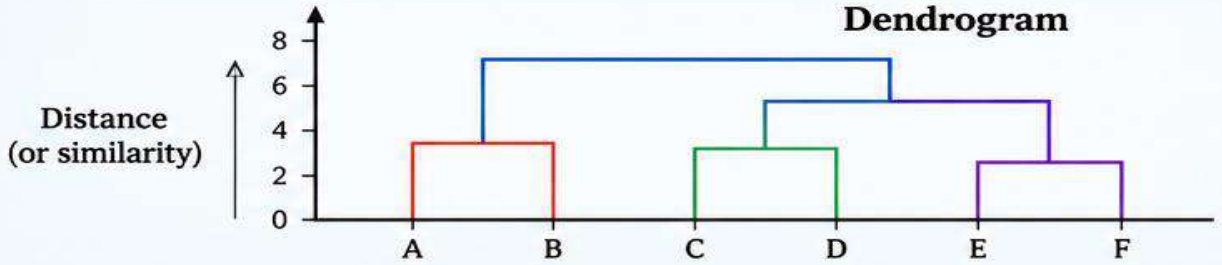
- Most widely used partitioning method.
- Divides data into K clusters.
- Assigns points based on distance to cluster centroids.
- Iteratively updates centroids to minimize within-cluster sum of squares (WCSS).

Fuzzy C-Means Clustering

- Extension of K-means.
- Allows a data point to belong to multiple clusters with different degrees of membership.
- Useful when clusters overlap.

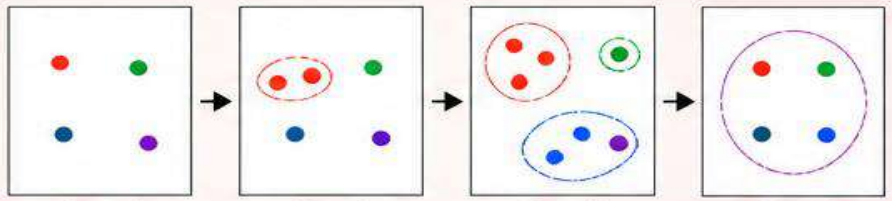
Hierarchical Clustering

- Hierarchical clustering uses a tree-like structure (dendrogram) to represent the nested grouping of data.
- It groups similar objects into clusters, either by merging smaller clusters or by splitting larger clusters.
- It does not require the number of clusters to be specified in advance.



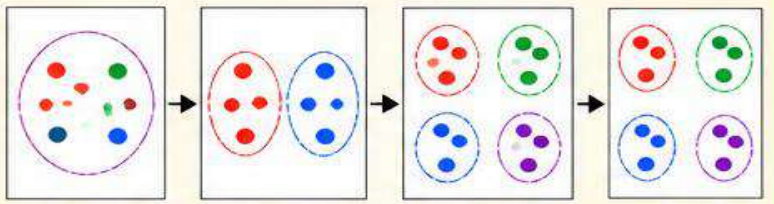
Agglomerative Clustering

- Bottom-up approach.
- Each element starts as a separate cluster.
- The two closest clusters are merged at each step.
- Continues until all elements are in a single cluster.
- Produces a dendrogram.



Divisive Clustering

- **Top-down approach.**
- Starts with the entire dataset as a single cluster.
- **Recursively splits the cluster into smaller clusters.**
- Continues until each element forms a cluster or the desired number of clusters is obtained.
- **Produces a dendrogram.**

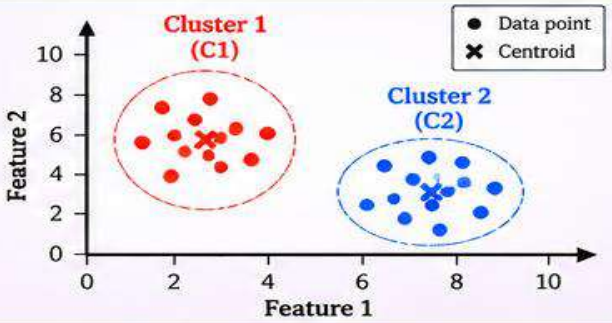


Partitioning Clustering

- Partitioning clustering divides the data into a predefined number of clusters by optimizing a **criterion** (e.g., minimizing within-cluster distance).

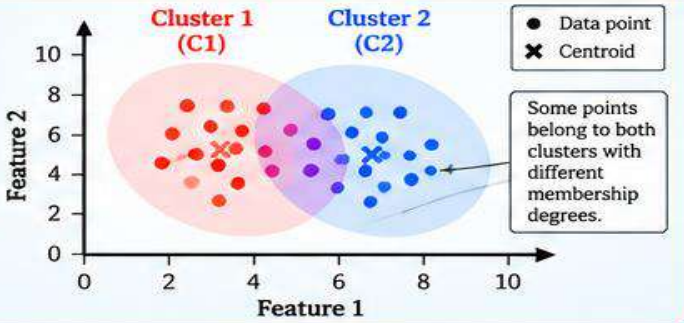
K-Means Clustering

- The data is divided into K clusters.
- Each cluster is represented by a centroid (mean).
- Objects are assigned to the nearest centroid based on distance (usually Euclidean distance).
- Centroids are updated iteratively until convergence.
- Each object belongs to only one cluster.



Fuzzy C-Means Clustering

- Similar to K-means but uses fuzzy membership.
- Each object can belong to more than one cluster with different degrees of membership.
- Membership values range from 0 to 1 and the sum of memberships for each object is 1.
- Useful for overlapping clusters.

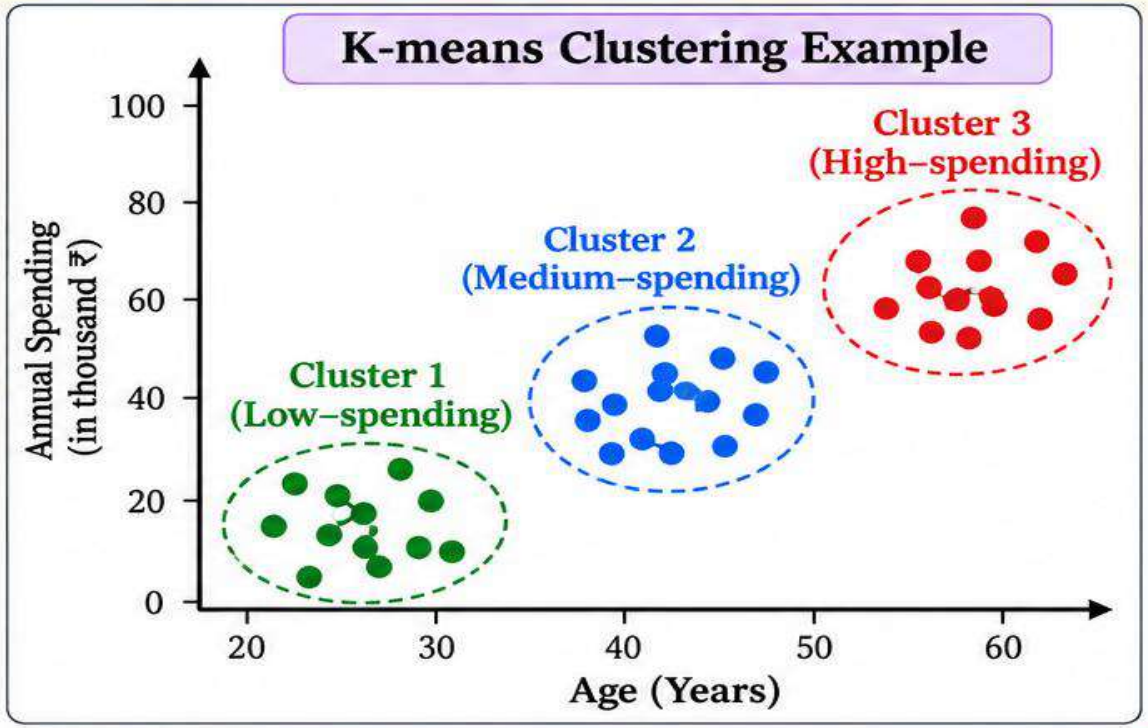


1. Introduction to K-means clustering

- **K-means** is an **unsupervised machine learning** algorithm used to divide a dataset into a fixed number of groups called **clusters**.
- The main idea is to group similar data points together and keep dissimilar data points in different clusters.
- The algorithm groups customers based on similarities in their data without requiring predefined class labels.

Example:
A supermarket wants to group customers according to their spending habits.

- **Cluster 1:** Low-spending customers
- **Cluster 2:** Medium-spending customers
- **Cluster 3:** High-spending customers



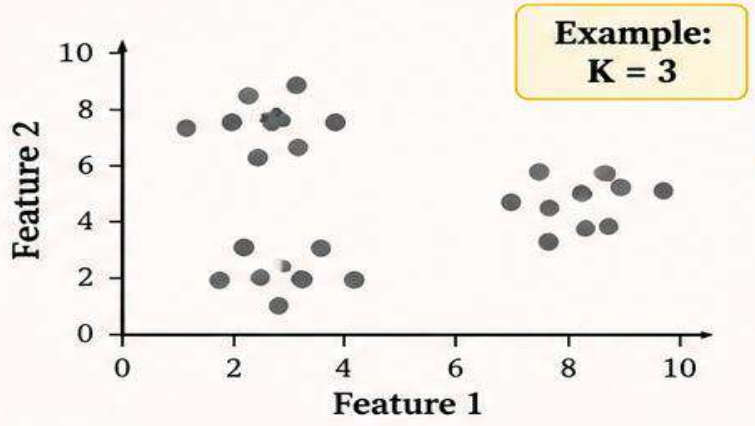
2. Important terms

Term	Explanation
K	Number of clusters to create.
Cluster	A group of similar data points.
Centroid	The center of a cluster, calculated using the mean of its points.
Distance	A measure of how far a data point is from a centroid.
Iteration	One round of assigning points and updating centroids.
Convergence	The point at which the clusters or centroids stop changing significantly.

3. Steps in the K-means algorithm

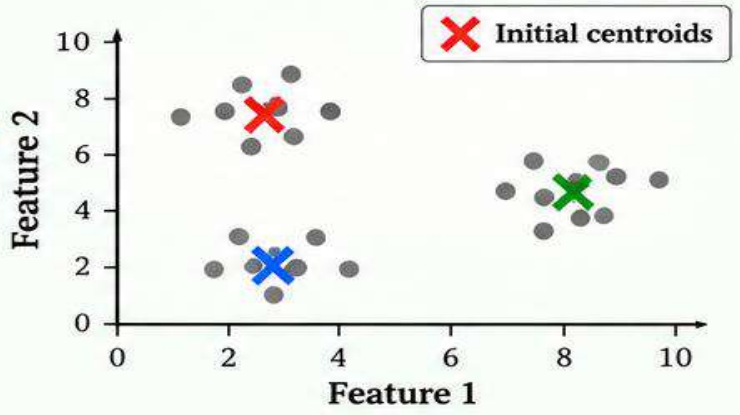
1 Step 1: Choose K

Decide the number of clusters.



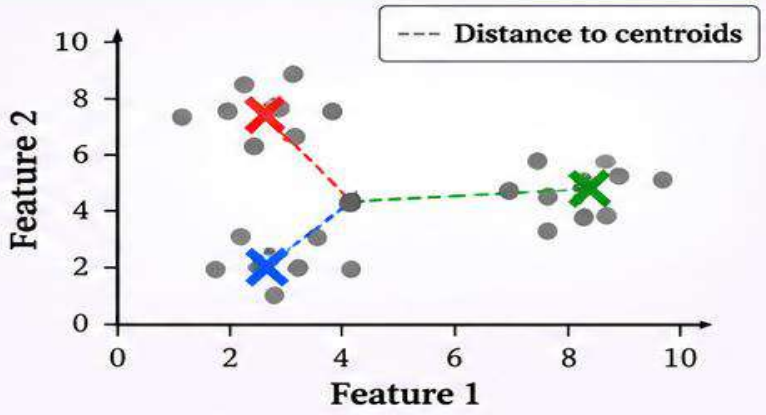
2 Step 2: Initialize centroids

Select **K** initial cluster centers.



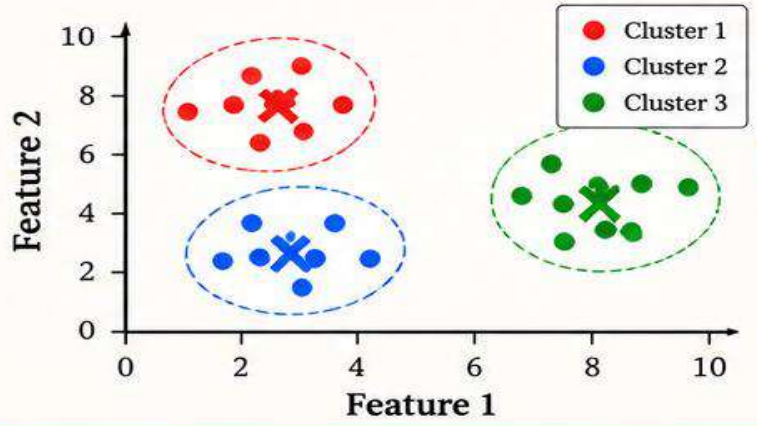
3 Step 3: Calculate distances

Measure the distance from every point to each centroid.



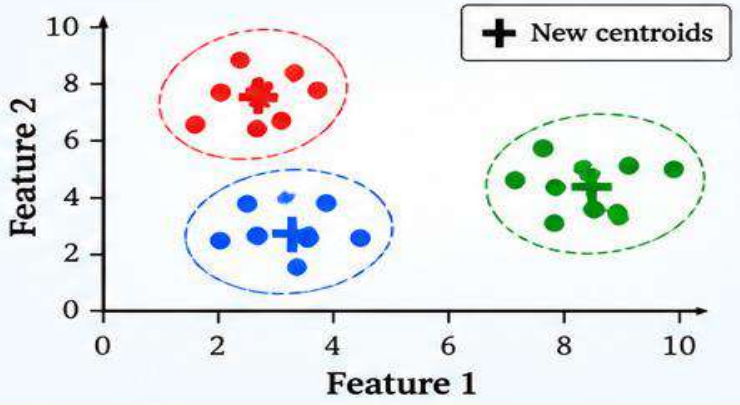
4 Step 4: Assign clusters

Assign each point to its nearest centroid.



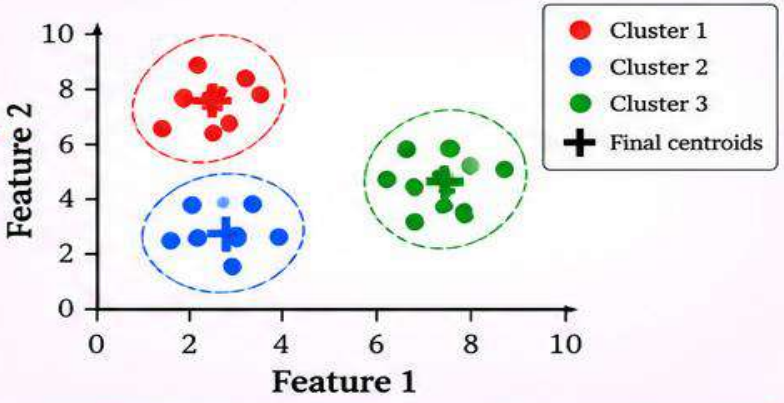
5 Step 5: Update centroids

Calculate the mean of the points in each cluster.



6 Step 6: Repeat

Repeat distance calculation, assignment **and** updating until **convergence.**



4. Using distance to assign clusters

- K-means commonly uses **Euclidean distance** to determine which centroid is closest to a data point.
- Each data point is assigned to the cluster with the **nearest centroid** (the smallest distance).

Euclidean distance formula

For two points $A = (x_1, y_1)$ and $B = (x_2, y_2)$:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

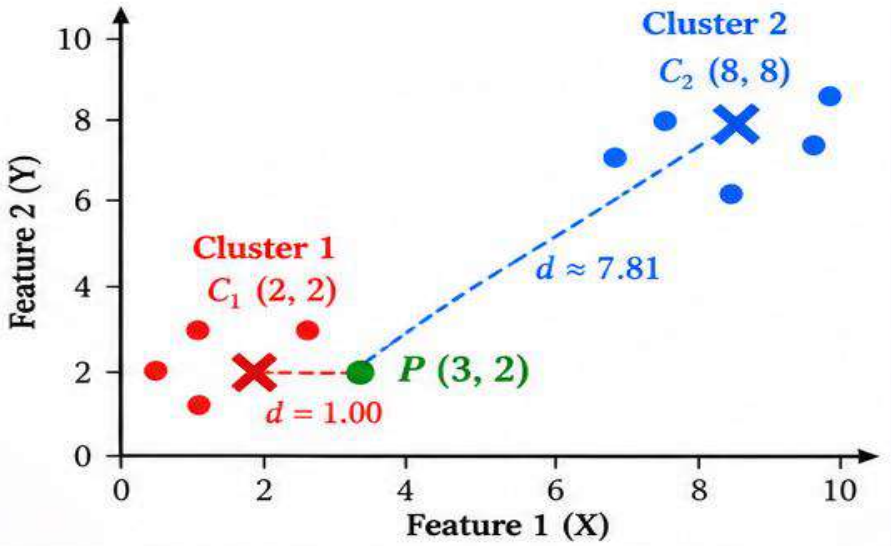
For a point with multiple features:

$$d(X, C) = \sqrt{\sum_{j=1}^{n} (x_j - c_j)^2}$$

Where:

- X = data point
- C = centroid
- n = number of features
- x_j, c_j = corresponding feature values

Illustration of distance to centroids



Example: Assigning a data point

Suppose there are two centroids:

$C_1 = (2, 2), \quad C_2 = (8, 8)$

A new data point is:

$P = (3, 2)$

Find the distance from P to each centroid and assign P to the nearest cluster.

Distance from P to C_1

$C_1 = (2, 2), \quad P = (3, 2)$

$$d(P, C_1) = \sqrt{(3-2)^2 + (2-2)^2} = \sqrt{1 + 0} = \mathbf{1.00}$$

Distance from P to C_2

$C_2 = (8, 8), \quad P = (3, 2)$

$$d(P, C_2) = \sqrt{(3-8)^2 + (2-8)^2} = \sqrt{25 + 36} = \sqrt{61} \approx \mathbf{7.81}$$

Comparison of distances

Centroid	Distance from P
$C_1 = (2, 2)$	1.00
$C_2 = (8, 8)$	7.81

Since the distance to C_1 is smaller, the point P (3, 2) is assigned to **Cluster 1.**

Rule: Assign each data point to the cluster with the **nearest** centroid.

5. Updating the cluster centroids

- After assigning all data points to clusters, K-means calculates a **new centroid** for each cluster.
- The new centroid is the **mean** of all data points assigned to that cluster.
- The centroid is the **average position** of the points in the cluster.

Centroid updating formula

For a two-dimensional dataset:

$$C = \left(\frac{\sum x_i}{m}, \frac{\sum y_i}{m} \right)$$

Where:

- C = new centroid
- m = number of data points in the cluster
- x_i, y_i = coordinates of each point in the cluster

Example

Suppose **Cluster 1** contains three points:

$$A = (1, 2), \quad B = (3, 2), \quad C = (2, 5)$$

Point	x	y
A	1	2
B	3	2
C	2	5

Mean of X-coordinates

$$\bar{x} = \frac{1+3+2}{3} = \frac{6}{3} = \mathbf{2}$$

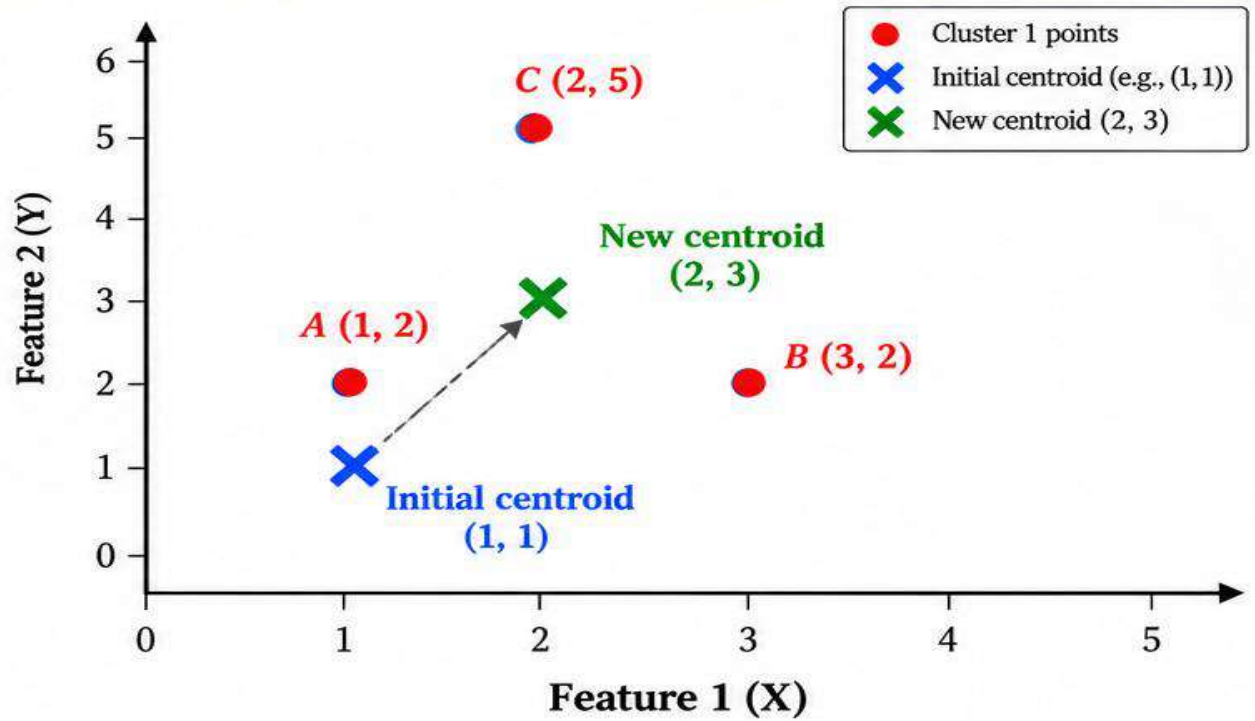
Mean of Y-coordinates

$$\bar{y} = \frac{2+2+5}{3} = \frac{9}{3} = \mathbf{3}$$

New centroid

$$C_1 = (\bar{x}, \bar{y}) = \mathbf{(2, 3)}$$

Update of centroid (visualization)



6. Complete worked example of K-means

1 Dataset

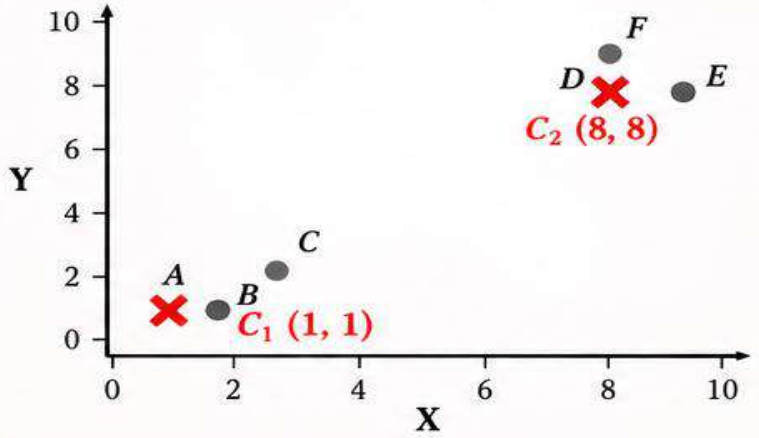
Consider six data points divided into $K = 2$ clusters.

Point	X	Y
A	1	1
B	2	1
C	3	2
D	8	8
E	9	8
F	8	9

2 Initial centroids

Assume the initial centroids are:

$$C_1 = (1, 1), \qquad C_2 = (8, 8)$$



3 Iteration 1: Assign the points

Calculate the distance from each point to both centroids and assign it to the nearest one.

Point	Distance to C_1 (1, 1)	Distance to C_2 (8, 8)	Cluster
A (1, 1)	0.00	9.90	1
B (2, 1)	1.00	9.22	1
C (3, 2)	2.24	7.81	1
D (8, 8)	9.90	0.00	2
E (9, 8)	10.63	1.00	2
F (8, 9)	10.63	1.00	2

- **Cluster 1** contains A, B and C.
- **Cluster 2** contains D, E and F.

4 Iteration 1: Calculate new centroids

New centroid for Cluster 1 (A, B, C)

$$C_1 = \left(\frac{1+2+3}{3}, \frac{1+1+2}{3}\right) = \left(\frac{6}{3}, \frac{4}{3}\right)$$

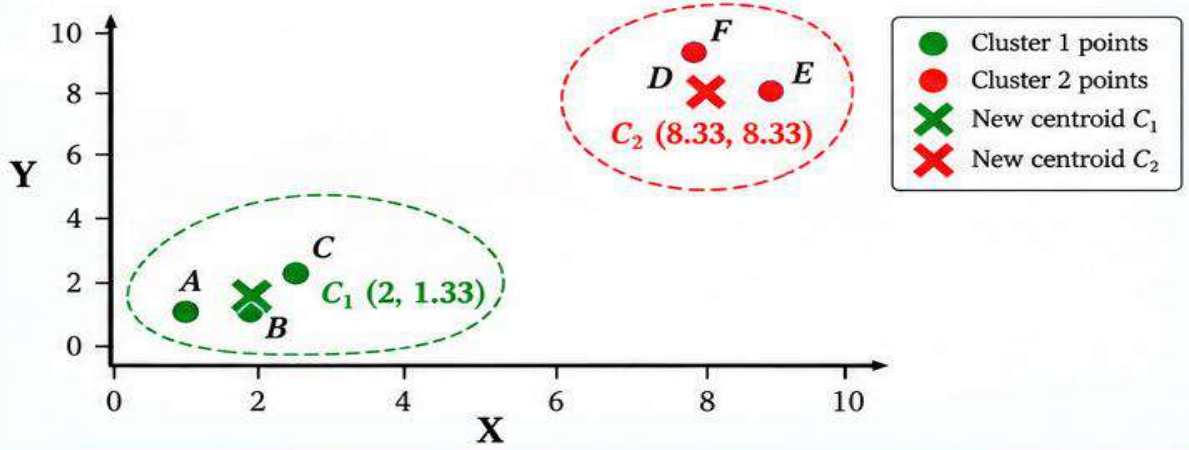
$$C_1 = (2, 1.33)$$

New centroid for Cluster 2 (D, E, F)

$$C_2 = \left(\frac{8+9+8}{3}, \frac{8+8+9}{3}\right) = \left(\frac{25}{3}, \frac{25}{3}\right)$$

$$C_2 = (8.33, 8.33)$$

Result after Iteration 1

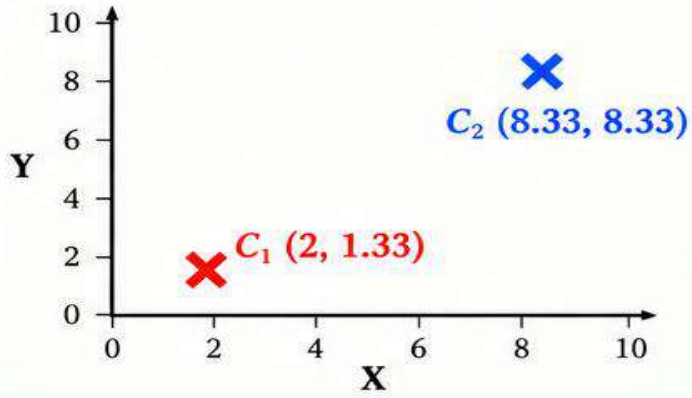


Iteration 2: Repeat the process

Using the updated **centroids** from **Iteration 1**, calculate the distances again and reassign all points.
In this example, **every** point remains **in its existing cluster**. The **centroids therefore remain unchanged, and the algorithm converges.**

Updated centroids from Iteration 1

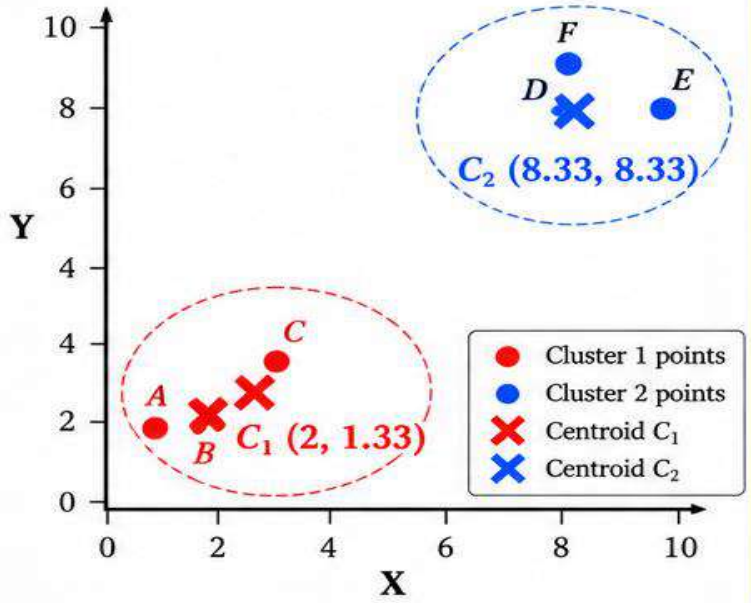
Cluster	Centroid (x, y)
C_1	(2, 1.33)
C_2	(8.33, 8.33)



Distance calculation and cluster assignment (Iteration 2)

Point (x, y)	Distance to centroid		Cluster
	C_1 (2, 1.33)	C_2 (8.33, 8.33)	
A (1, 1)	1.07	10.39	1
B (2, 1)	0.33	9.67	1
C (3, 2)	1.07	8.48	1
D (8, 8)	8.48	0.47	2
E (9, 8)	9.24	0.75	2
F (8, 9)	9.24	0.75	2

Clusters after Iteration 2



Final clusters (after convergence)

Cluster 1

A (1, 1), *B* (2, 1), *C* (3, 2)

Centroid: (2, 1.33)

Cluster 2

D (8, 8), *E* (9, 8), *F* (8, 9)

Centroid: (8.33, 8.33)

Conclusion

- Every point remains in its existing cluster.
- The centroids remain unchanged.
- The algorithm has **converged.**

Final result:

Cluster 1 = {A, B, C} **Cluster 2 = {D, E, F}**

7. Choosing the number of clusters

Overview

- The value of K must be selected before running K-means.
- Choosing a suitable number of clusters is important because too few clusters may combine dissimilar points, while too many clusters may divide natural groups unnecessarily.
- Two common techniques are the **Elbow method** and the **Silhouette score**.

Elbow method

The Elbow method helps identify a suitable value of K by examining how the within-cluster sum of squares (WCSS) changes as the number of clusters increases.

Within-cluster sum of squares (WCSS)

$$WCSS = \sum_{k=1}^{K} \sum_{X_i \in C_k} \|X_i - \mu_k\|^2$$

Where:

- X_i is a data point.
- μ_k is the centroid of its assigned cluster.
- C_k is the kth cluster.
- $WCSS$ measures the total squared distance of points from their cluster centroid. A smaller WCSS indicates more compact clusters.

Procedure

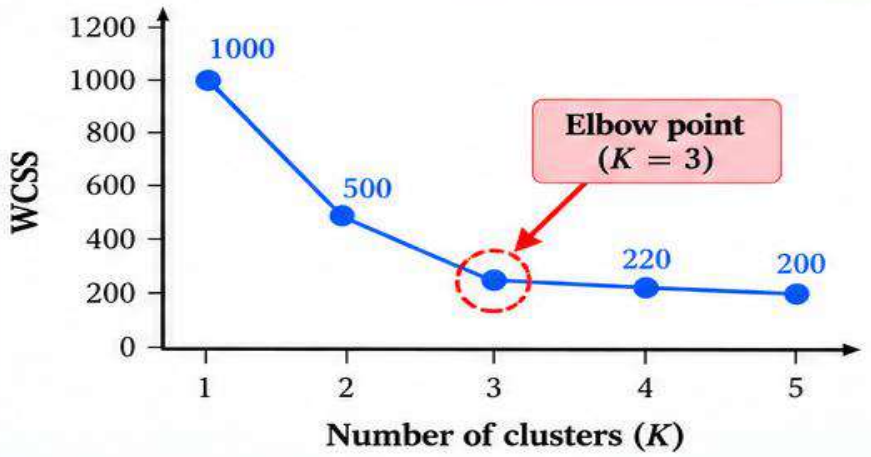
1. Run K-means with $K = 1$.
2. Calculate the WCSS.
3. Repeat with $K = 2, 3, 4, \ldots$.
4. Plot K on the X-axis and WCSS on the Y-axis.
5. Select a reasonable value near the point where the curve begins to flatten (the elbow point).

Example

Suppose we run K-means for different values of K and obtain the following WCSS values:

Number of clusters (K)	WCSS
1	1000
2	500
3	250
4	220
5	200

Elbow plot



Interpretation

- The reduction in WCSS is large from $K = 1$ to $K = 3$.
- After $K = 3$, the reduction becomes smaller.
- Therefore, $K = 3$ is a **reasonable choice** in this example.
- The elbow is a guideline; sometimes the graph does not show a clear elbow.

Key point

Choose the number of clusters K at the **"elbow"** where further increase in K gives only a small improvement in WCSS.

Silhouette score

- The Silhouette score measures how well a data point fits into its own cluster compared with other clusters.
- It measures the separation between clusters and helps in choosing a suitable number of clusters (K).
- The score ranges from -1 to $+1$.

Silhouette score formula

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$ = average distance from the point to other points in its own cluster.
- $b(i)$ = lowest average distance from the point to points in another cluster.

Interpretation of silhouette score

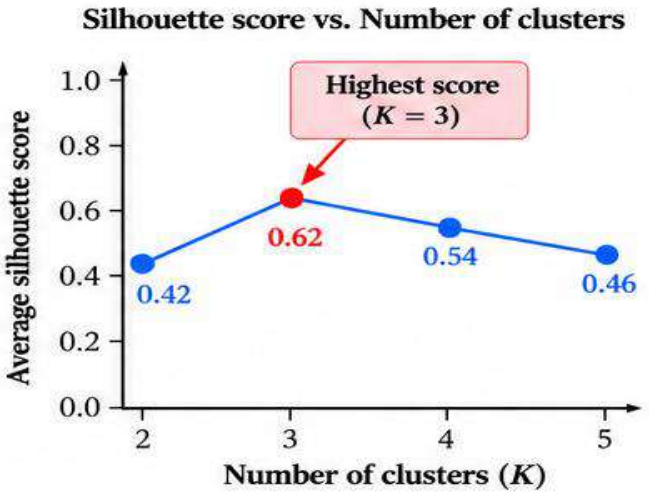
Score	Meaning
Near +1	Point is well matched to its cluster
Near 0	Point is near a cluster boundary
Below 0	Point may be assigned to the wrong cluster

- Calculate the average silhouette score for different values of K. A **higher score** generally indicates **better-separated clusters.**

Example: Silhouette scores for different values of K

Suppose we run K-means for different values of K and obtain the following average silhouette scores.

Number of clusters (K)	Average silhouette score
2	0.42
3	**0.62**
4	0.54
5	0.46



Interpretation

- The highest average silhouette score is obtained for $K = 3$.
- Therefore, $K = 3$ is a **suitable choice** as it gives better-separated clusters in this example.
- A **higher** silhouette score indicates that the clusters are compact and well separated.

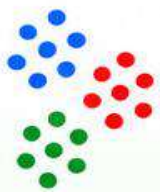
Other factors for choosing K

1 Domain knowledge



- Choose clusters that make practical sense for the application.
- Use background knowledge of the data and problem domain.

2 Cluster size



- Check whether the groups are meaningful.
- Avoid choosing a K that creates very small or unnecessarily large clusters.

3 Stability



- Check whether similar clusters appear with different initializations.
- A good choice of K should give stable and consistent cluster results.

4 Interpretability



- Select a clustering result that is easy to interpret and useful for the application.
- The clusters should provide meaningful and actionable insights.

Advantages and limitations of K-means

Advantages

1. Simple to understand and implement.
2. Efficient for many numerical datasets.
3. Easy to interpret.
4. Works well for compact, well-separated clusters.

Limitations

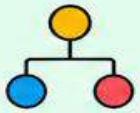
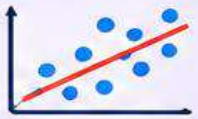
1. Requires the number of clusters to be specified in advance.
2. Results can depend on the initial centroids.
3. Sensitive to outliers.
4. Distance calculations can be dominated by features with large scales.
5. Does not naturally identify clusters with irregular shapes.

Evaluating model performance

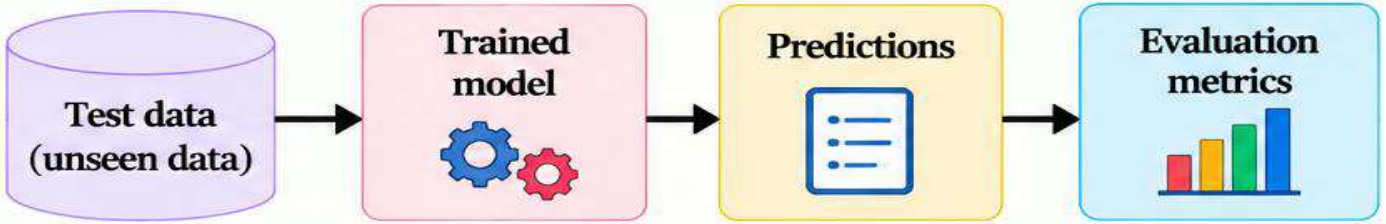
Introduction

- Model evaluation measures how well a machine learning model performs on data that was not used to train it.
- It helps to understand how accurately the model can make predictions on new, unseen data.
- Different evaluation methods are used for different types of machine learning problems.
- The choice of evaluation metric depends on whether the problem is classification, clustering or regression.

Evaluation methods for different problem types

Problem type	Common evaluation methods
 **Classification**	Confusion matrix, precision, recall, F-measure, ROC-AUC
 **Clustering**	Silhouette score, inertia, cluster separation
 **Regression**	MAE, MSE, RMSE, R^2

Model evaluation process



Data that was not used for training the model

Use the trained model to make predictions

Obtain predicted results from the model

Calculate suitable evaluation metrics based on the problem type

Note on binary classification

- The metrics in the following sections mainly apply to binary classification.
- In binary classification, the model predicts one of two classes, such as Positive or Negative.
- Examples:
  - Disease: Positive (diseased) or Negative (not diseased)
  - Email: Positive (spam) or Negative (not spam)
- The meaning of positive and negative depends on the application.
- Always identify which class is treated as positive before calculating the evaluation metrics.

Confusion matrix

Definition

- A confusion matrix is a table used to evaluate a classification model by comparing the actual class labels with the predicted class labels.
- It shows how many predictions are correct and how many are incorrect.

Example

Consider a model that predicts whether a student will pass or fail an examination.

- **Positive class :** Pass
- **Negative class :** Fail
- Suppose the model predicts the results of 100 students.

Confusion matrix for the example

		Predicted class	
		Predicted Pass (Positive)	**Predicted Fail (Negative)**
Actual class	**Pass (Positive)**	**40 (TP)**	**10 (FN)**
	Fail (Negative)	**5 (FP)**	**45 (TN)**

The four values in the confusion matrix

TP — True Positive (TP)

Value: 40

Students who actually **passed** and were **predicted to pass.**

FN — False Negative (FN)

Value: 10

Students who actually **passed** but were **predicted to fail.**

FP — False Positive (FP)

Value: 5

Students who actually **failed** but were **predicted to pass.**

TN — True Negative (TN)

Value: 45

Students who actually **failed** and were **predicted to fail.**

Understanding the four values

- **True Positive (TP):** The model correctly predicts the positive class.
- **True Negative (TN):** The model correctly predicts the negative class.
- **False Positive (FP):** The model incorrectly predicts positive for a negative case.
- **False Negative (FN):** The model incorrectly predicts negative for a positive case.

Important note

- The meaning of positive and negative depends on the application.
- Always identify which class is treated as positive before calculating the evaluation metrics.

Evaluation Metrics for Classification Model

Accuracy

Definition

Accuracy measures the proportion of total predictions that are correct.

Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Using the example

$$\text{Accuracy} = \frac{40 + 45}{40 + 45 + 5 + 10}$$

$$= \frac{85}{100} = 0.85$$

Accuracy = 85%

Interpretation

- The model correctly predicted the examination results of 85 out of 100 students.

Limitation

- Accuracy can be misleading when one class is much more common than another. For example, a model that predicts every student as Pass could achieve high accuracy if nearly all students pass.

Precision

Definition

Precision measures how many of the cases predicted as positive are actually positive.
It answers the question: Of all the cases predicted as positive, how many were correct?

Formula

$$\text{Precision} = \frac{TP}{TP + FP}$$

Using the example

$$\text{Precision} = \frac{40}{40 + 5} = \frac{40}{45}$$

Precision = 88.89%

Interpretation

- Out of 45 students predicted to pass, approximately 88.89% actually passed.

When is precision useful?

- Precision is important when false positive predictions are costly.
- Example: In email spam detection, high precision means that most emails classified as spam really are spam.

Recall

Definition

Recall measures how many of the actual positive cases were correctly identified by the model.
It answers the question: Of all the cases that were actually positive, how many did the model identify correctly?

Formula

$$\text{Recall} = \frac{TP}{TP + FN}$$

Using the example

$$\text{Recall} = \frac{40}{40 + 10} = \frac{40}{50}$$

Recall = 80%

Interpretation

- The model correctly identified 80% of all students who actually passed.

When is recall useful?

- Recall is important when missing a positive case is costly.
- Example: In medical screening, high recall helps identify a greater proportion of people who may have a condition.

Sensitivity and Specificity

Sensitivity

- Sensitivity is another name for recall or the true positive rate.

Formula

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Using the example

$$\text{Sensitivity} = \frac{40}{40 + 10} = 80\%$$

Interpretation

- It measures the ability of the model to correctly identify actual positive cases.

Specificity

- Specificity measures how many actual negative cases were correctly identified.

Formula

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Using the example

$$\text{Specificity} = \frac{45}{45 + 5} = 90\%$$

Interpretation

- It measures the ability of the model to correctly identify actual negative cases.

Difference between sensitivity and specificity

Sensitivity	Specificity
Measures correct identification of positive cases	Measures correct identification of negative cases
Uses TP and FN	Uses TN and FP
Also called recall	Also called true negative rate
High sensitivity means fewer false negatives	High specificity means fewer false positives

Example in disease screening

Suppose a screening model identifies whether a person may have a disease.

- High sensitivity means fewer people with the disease are missed.
- High specificity means fewer people without the disease are incorrectly flagged.

The suitable balance depends on the purpose of the screening test and the consequences of errors.

F1-score and ROC Curve

F-measure or F1-score

Definition

- The F-measure combines precision and recall into a single evaluation metric.
- The most commonly used version is the F1-score, which is the harmonic mean of precision and recall.

Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Using the example

$$\text{Precision} = 0.8889, \qquad \text{Recall} = 0.80$$

$$F1 = 2 \times \frac{0.8889 \times 0.80}{0.8889 + 0.80}$$

F1 ≈ 84.21%

Interpretation

- The F1-score summarizes the balance between precision and recall.

Why is the F1-score useful?

Suppose two models have the following results:

Model	Precision	Recall	F1-score
Model A	95%	60%	73.55%
Model B	80%	80%	80%

- Model A has higher precision, but Model B has a better balance between precision and recall.
- Therefore, Model B has the higher F1-score.

Note

- F1-score does not include true negatives directly.
- When performance on negative cases is important, specificity should also be considered.

ROC curve

Definition

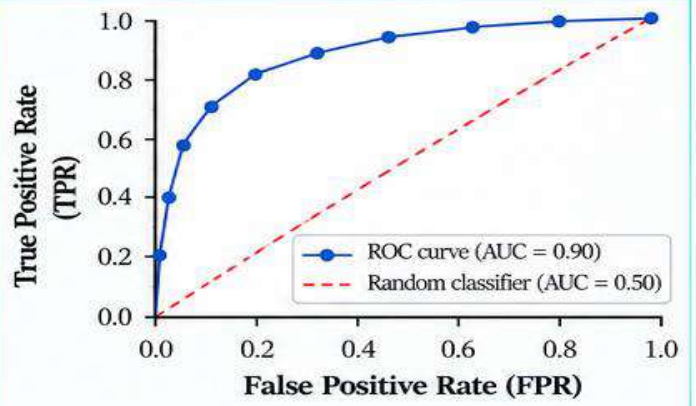
- ROC stands for Receiver Operating Characteristic.
- An ROC curve is a graph used to evaluate the performance of a binary classification model at different classification thresholds.
- It plots:
  - **X-axis:** False Positive Rate (FPR).
  - **Y-axis:** True Positive Rate (TPR), which is the same as sensitivity or recall.

Formulas

$$TPR = \frac{TP}{TP + FN}$$
(True Positive Rate)

$$FPR = \frac{FP}{FP + TN}$$
(False Positive Rate)

Example ROC curve



What is a classification threshold?

- A classification model may predict a probability rather than a class directly.
- For example, suppose a model predicts a 75% probability that a person belongs to the positive class.
- At a threshold of 0.50, the prediction is positive.
- At a threshold of 0.80, the prediction is negative.
- Changing the threshold changes the number of positive and negative predictions.
- The ROC curve shows the resulting changes in TPR and FPR.

Interpretation of the ROC curve

- A curve closer to the top-left corner generally indicates better classification performance.
- A curve near the diagonal represents performance similar to random ranking.
- The curve illustrates the trade-off between identifying positive cases and producing false positives.

Area under the curve (AUC)

- AUC summarizes how well the model ranks positive cases above negative cases.

AUC value	General interpretation
1.00	Perfect ranking
0.90 – 0.99	Excellent discrimination
0.80 – 0.89	Good discrimination
0.70 – 0.79	Moderate discrimination
0.50	Random ranking, on average

These ranges are general guidelines rather than universal rules.

Advantages of ROC-AUC

1. Evaluates classification performance across different thresholds.
2. Helps compare different classification models.
3. Measures how well a model distinguishes between two classes.
4. Does not require selecting one particular classification threshold.

Limitations of ROC-AUC

1. Does not determine the best threshold by itself.
2. May provide an incomplete picture for highly imbalanced datasets.
3. Does not directly show the real-world cost of false positives and false negatives.

For highly imbalanced datasets, a precision-recall curve can provide useful additional information.

Cross-validation

Definition

- Cross-validation is a model evaluation technique used to estimate how well a machine learning model performs on unseen data. It divides the available dataset into different subsets and evaluates the model multiple times using different training and validation sets. It helps us understand whether a model can generalize to new data rather than simply memorizing the training data.

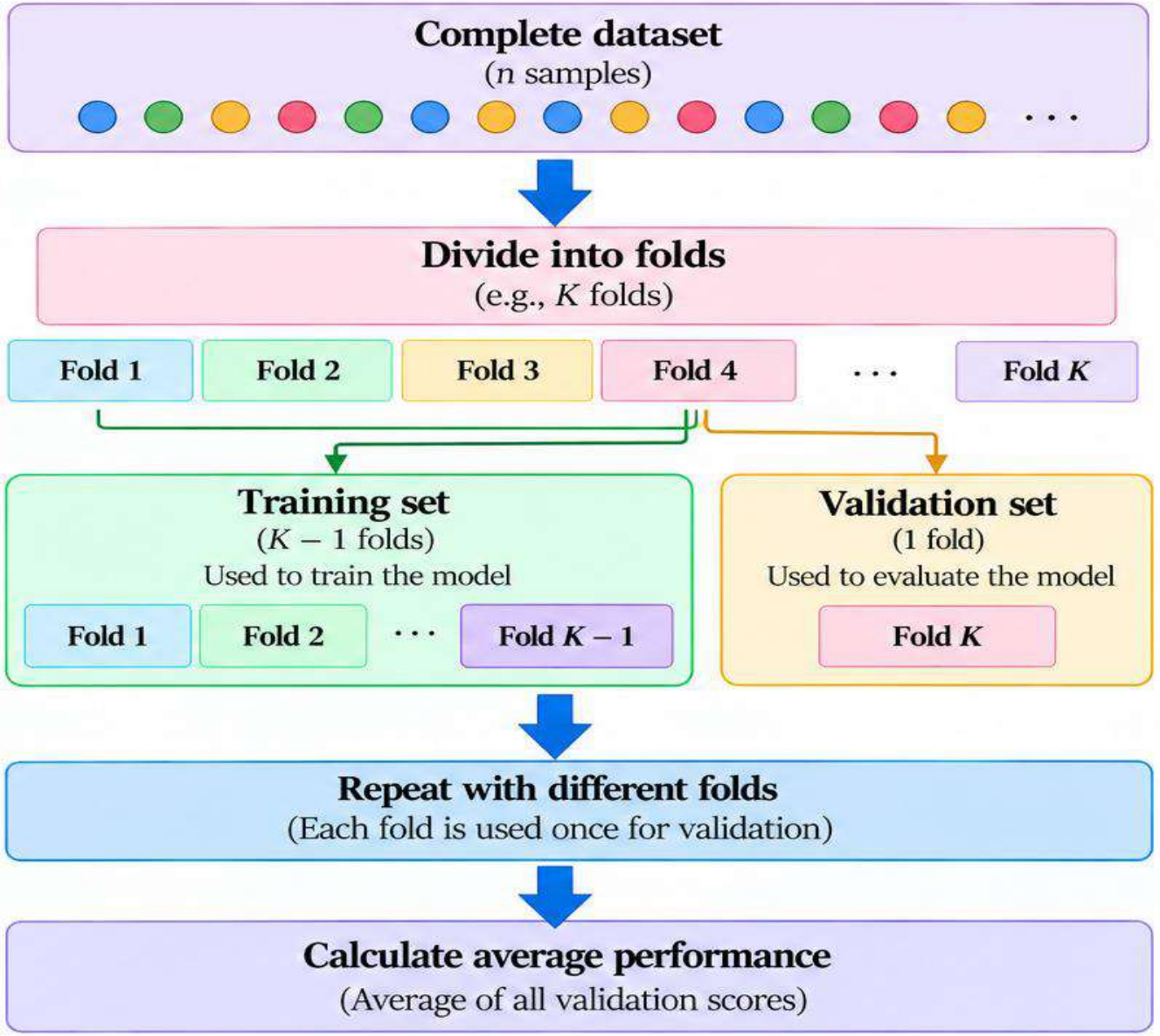
Objectives of cross-validation

- Estimate the generalization performance of a model.
- Compare different machine learning algorithms.
- Select suitable hyperparameters.
- Identify variations in performance across different data splits.
- Reduce dependence on a single validation split.

How cross-validation works

1. Divide the dataset into multiple subsets.
2. Select one subset for validation.
3. Use the remaining subsets to train the model.
4. Evaluate the trained model on the validation subset.
5. Repeat the process using different validation subsets.
6. Calculate the average performance across all runs.

Cross-validation process



K-fold cross-validation

Definition

K-fold cross-validation divides a dataset into K approximately equal-sized subsets called folds. The model is trained K times, using one fold for validation and the remaining $K - 1$ folds for training during each run.

Steps

1. Divide the dataset into K folds.
2. Select the first fold for validation.
3. Train the model on the remaining folds.
4. Record the validation score.
5. Repeat until every fold has been used for validation once.
6. Calculate the average validation score.

Example: 5-fold cross-validation

- Suppose a dataset contains 100 student records.
- Choose $K = 5$.
- Each fold contains approximately 20 records ($100 \div 5 = 20$).

Dataset (100 records)				
Fold 1 (20 records)	**Fold 2** (20 records)	**Fold 3** (20 records)	**Fold 4** (20 records)	**Fold 5** (20 records)

Five-fold cross-validation process

Run 1	Run 2	Run 3	Run 4	Run 5
Validation: Fold 1	Validation: Fold 2	Validation: Fold 3	Validation: Fold 4	Validation: Fold 5
F1 Valid.	F1 Train	F1 Train	F1 Train	F1 Train
F2 Train	F2 Valid.	F2 Train	F2 Train	F2 Train
F3 Train	F3 Train	F3 Valid.	F3 Train	F3 Train
F4 Train	F4 Train	F4 Train	F4 Valid.	F4 Train
F5 Train	F5 Train	F5 Train	F5 Train	F5 Valid.

Training fold Validation fold

In each run:

- **Training data:** 80 records (80%).
- **Validation data:** 20 records (20%).

- Across all five runs, every record is used once for validation and four times for training.
- This helps to obtain a more reliable estimate of the model's performance.

Calculating average accuracy

Suppose the five validation accuracies are 90%, 92%, 88%, 91% and 89%.

$$\text{Average accuracy} = \frac{90 + 92 + 88 + 91 + 89}{5} = \frac{450}{5} = \mathbf{90\%}$$

The average cross-validation accuracy is 90%.

Types of cross-validation

Type	Description
K-fold	Divides the dataset into K folds.
Stratified K-fold	Maintains approximately the same class proportions in each fold; useful for classification.
Leave-one-out	Uses one observation for validation and all remaining observations for training.
Repeated K-fold	Repeats K-fold validation with different data partitions.

Advantages

1. Provides a more reliable performance estimate than a single split in many situations.
2. Helps compare models and select hyperparameters.
3. Makes efficient use of limited data.
4. Reveals variation in performance across different folds.

Limitations

1. Requires training the model multiple times.
2. Can be computationally expensive for large datasets.
3. Ordinary random K-fold is not suitable for every problem, such as time-series data that must preserve temporal order.
4. Data leakage can lead to misleading results if preprocessing is not performed correctly within each training fold.

Bootstrap Sampling

Definition

- Bootstrap sampling is a statistical resampling technique in which multiple samples are generated from the original dataset by randomly selecting observations with replacement.
- It is used to estimate the variability and uncertainty of a statistical measure or a machine learning model's performance.

What does "with replacement" mean?

- After selecting an observation, it is returned to the dataset before the next selection.
- Therefore, an observation may appear multiple times in a bootstrap sample, while other observations may not appear at all.

Example

Suppose the original dataset contains five student marks:

$$D = \{10, 20, 30, 40, 50\}$$

A bootstrap sample of size five might be:

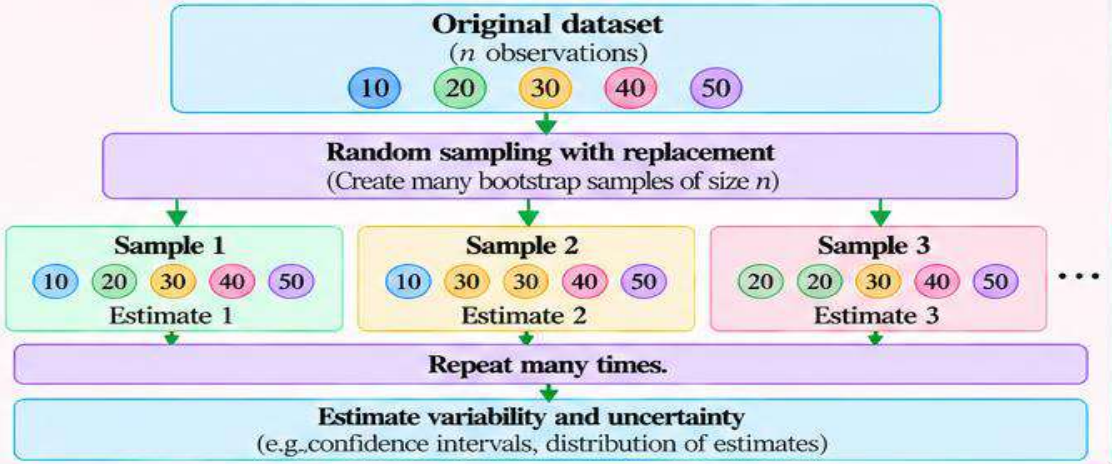
$$D_1 = \{10, 20, 20, 40, 50\}$$

Notice that:

- The value 20 appears twice.
- The value 30 is not selected.
- The bootstrap sample still contains five observations.
- This is possible because sampling is performed with replacement.

Steps in bootstrap sampling

1. Start with the original dataset containing n observations.
2. Randomly select n observations with replacement.
3. Calculate a statistic or train and evaluate a model using the sample.
4. Repeat the sampling process many times.
5. Examine the resulting estimates to understand their variability and uncertainty.



Numerical example: Estimating the mean

Original dataset:

$D = \{10, 20, 30, 40, 50\}$

Original mean:

$$\bar{x} = \frac{10+20+30+40+50}{5} = 30$$

Bootstrap sample	Values	Sample mean
Sample 1	10, 20, 20, 40, 50	28
Sample 2	10, 30, 30, 40, 50	32
Sample 3	20, 20, 30, 40, 50	32

- The means vary because each bootstrap sample contains a different combination of observations.

Average of three bootstrap means $= \frac{28 + 32 + 32}{3} = 30.67$

Bootstrap sampling for model evaluation

Bootstrap sampling can also estimate the uncertainty in a model's performance. For example, to estimate the variability of a classifier's accuracy:

1. Generate a bootstrap sample from the available data.
2. Train and evaluate the model using a properly designed resampling procedure.
3. Record the accuracy.
4. Repeat the process many times.
5. Examine the distribution of accuracy estimates and, where appropriate, calculate a confidence interval.

Important: Evaluating a model on the same observations used to train it can produce overly optimistic results. For model evaluation, use an appropriate out-of-bag or bootstrap validation procedure.

Out-of-bag observations

- Because bootstrap sampling uses replacement, some original observations are not selected in a particular sample.
- These are called out-of-bag (OOB) observations for that sample.
- They can be used to evaluate predictions from a model trained on the corresponding bootstrap sample.
- For example, Random Forests can estimate performance using OOB observations when oob_score = True is enabled with a suitable configuration.

Advantages

1. Helps estimate standard errors and confidence intervals.
2. Does not require assuming a specific parametric distribution.
3. Can be applied to many statistics and model-performance measures.
4. Is useful when theoretical uncertainty calculations are difficult.

Limitations

1. Repeated sampling and model training can be computationally expensive.
2. Results depend on the quality and representativeness of the original dataset.
3. May be unreliable for some small, highly biased or dependent datasets.
4. Requires a suitable resampling design for time-series or grouped observations.

Improving Model Performance

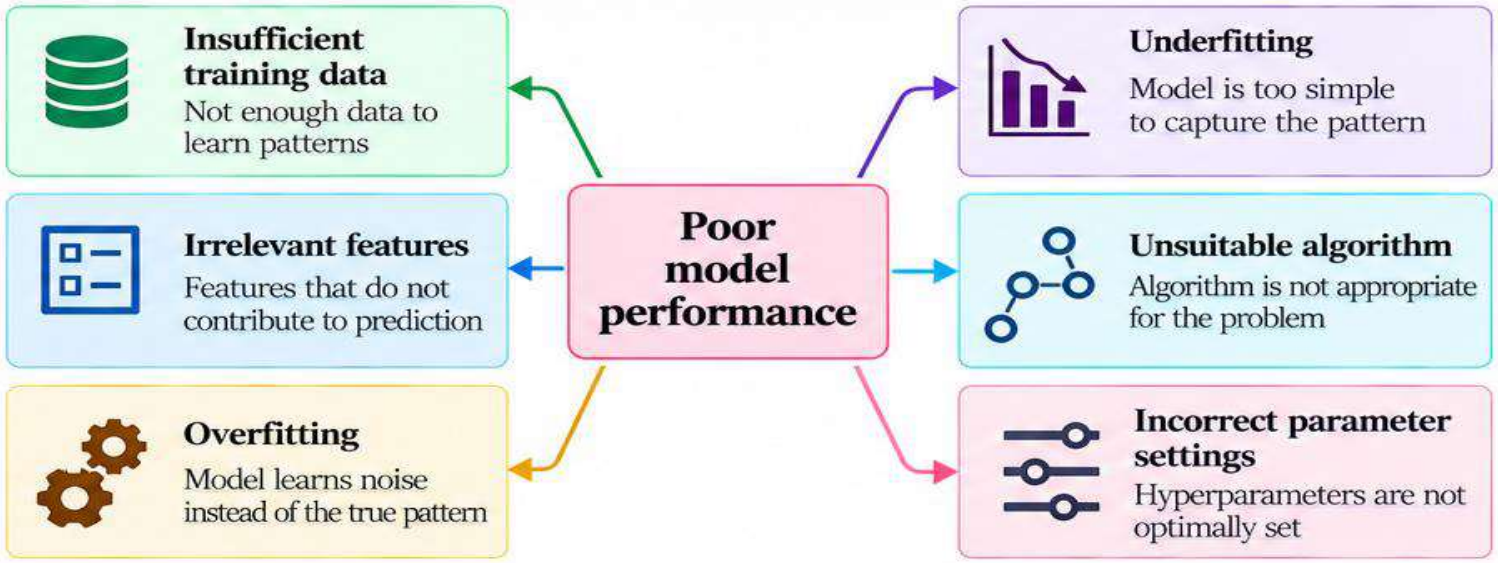
Introduction

- Improving model performance is the process of modifying a machine learning model, its training data, or its parameters to make predictions more accurate and reliable on unseen data.
- A model may perform poorly because of insufficient training data, irrelevant features, overfitting, underfitting, unsuitable algorithms, or incorrect parameter settings.

Example: A decision tree model predicts whether students will pass or fail an examination with 75% accuracy. By tuning its parameters, improving the quality of the training data, or using an ensemble method, we may improve its predictive performance.

Note: However, improvement must be confirmed using proper validation and testing; it is not guaranteed by changing the model.

Reasons for poor model performance



Common techniques for improving model performance

1 Collect more high-quality data

More relevant and diverse data can help the model learn better patterns.

2 Clean and preprocess data

Handle missing values, remove noise and correct errors in the data.

3 Feature selection and scaling

Select useful features and apply scaling (such as standardization or normalization) to numerical features when appropriate.

4 Choose a suitable algorithm

Select an algorithm that fits the nature of the problem and the data.

5 Tune hyperparameters

Use techniques such as grid search, random search or optimization methods to find better parameter settings.

6 Use cross-validation

Apply techniques such as K-fold cross-validation to get a reliable estimate of model performance.

7 Reduce overfitting and underfitting

Use techniques such as regularization, pruning, dropout or more complex models as needed.

8 Apply ensemble learning techniques

Use methods such as bagging, boosting and random forests to improve prediction accuracy and stability.

Bagging (Bootstrap Aggregating)

Definition

- Bagging stands for Bootstrap Aggregating.
- It is an ensemble learning technique in which multiple models are trained independently using different bootstrap samples drawn from the original training dataset.
- Each bootstrap sample is created by randomly selecting observations with replacement.

Working procedure

1. Start with the original training dataset.
2. Generate several bootstrap samples.
3. Train a separate model on each sample.
4. Obtain predictions from all trained models.
5. Combine the predictions to produce the final result.

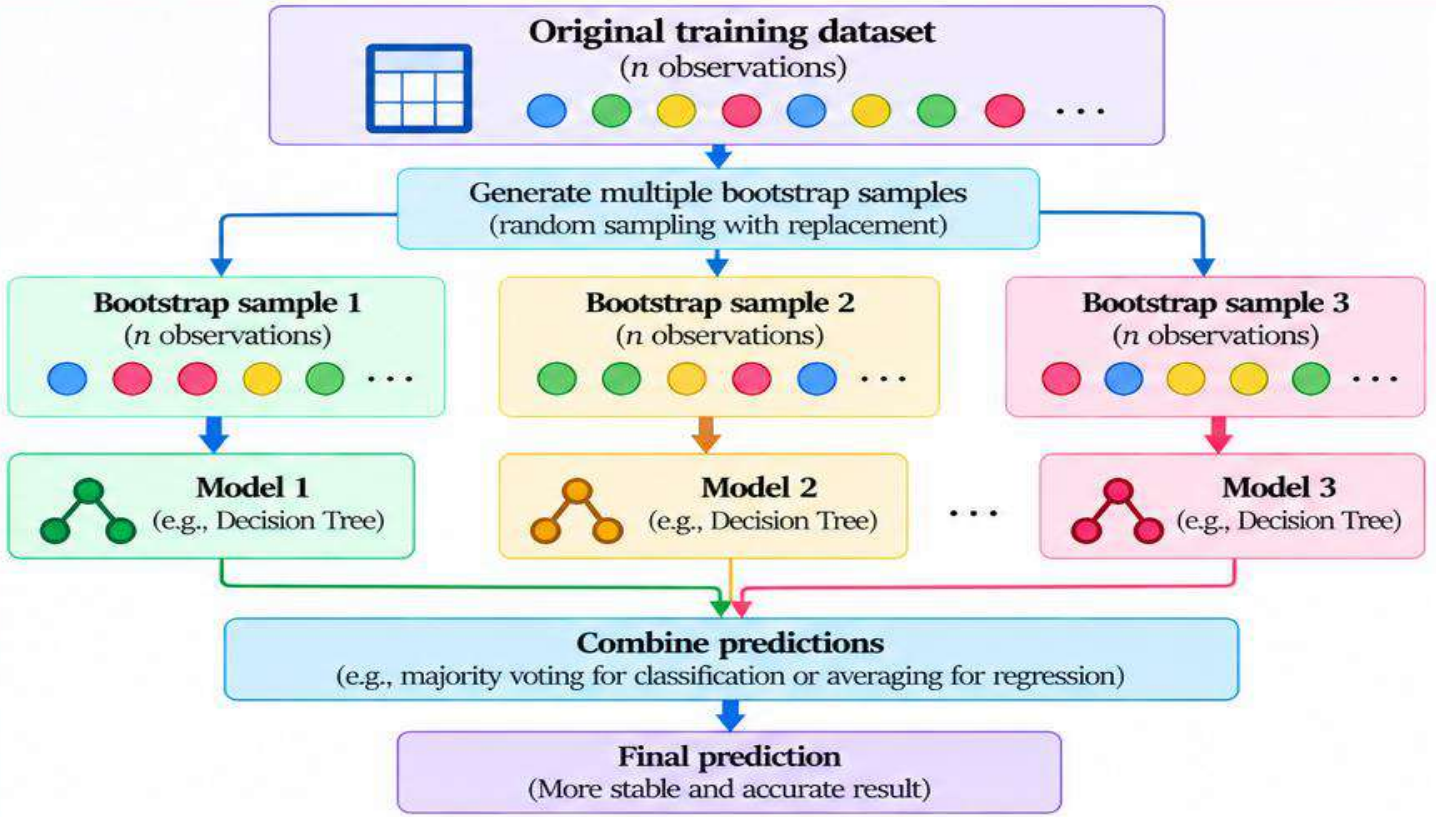
Example

Suppose five decision trees predict whether a customer will purchase a product.

Decision tree	Prediction
Tree 1	Buy
Tree 2	Buy
Tree 3	Not Buy
Tree 4	Buy
Tree 5	Not Buy

Three trees predict Buy and two predict Not Buy. By majority voting, the final prediction is **Buy**.

How bagging works



Advantages

1. Reduces variance and may reduce overfitting.
2. Improves the stability of predictions.
3. Models can usually be trained in parallel.
4. Works well with unstable learners such as decision trees.

Limitations

1. Requires more memory and computation than a single model.
2. May provide limited improvement if the models make highly correlated errors.
3. The final model is less interpretable than a single decision tree.

Boosting

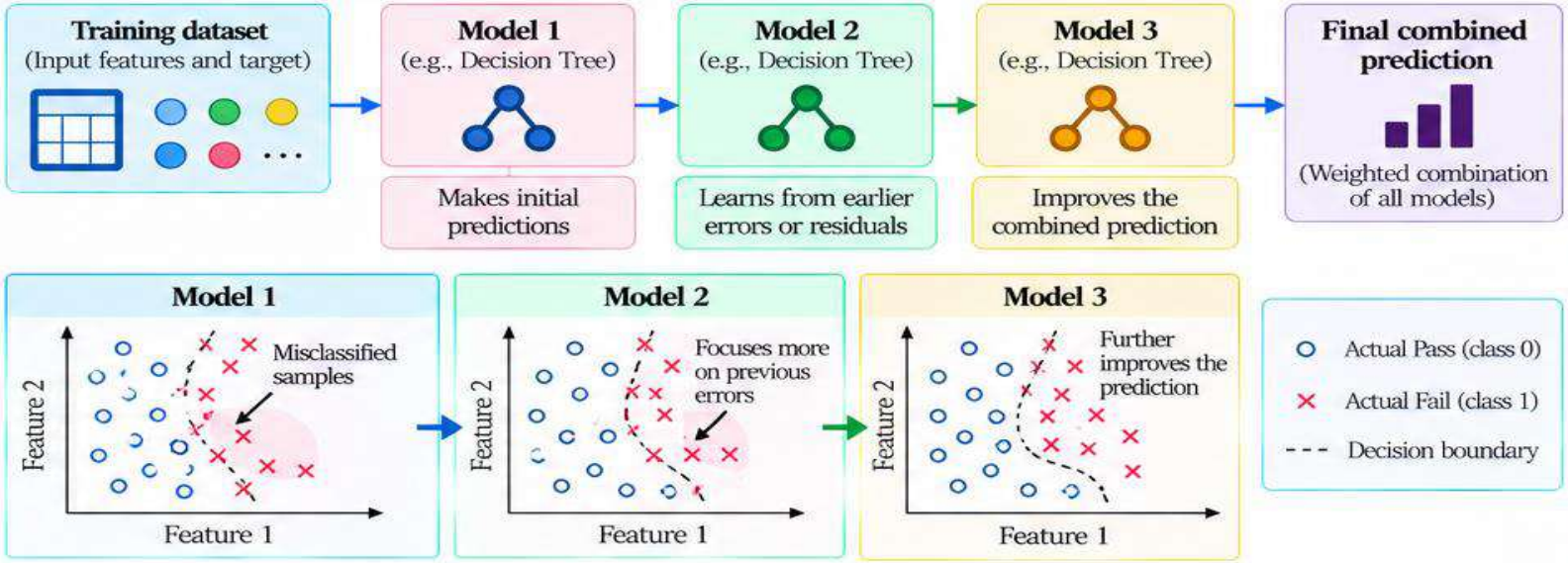
Definition

- Boosting is an ensemble learning technique that trains models sequentially, where each new model attempts to improve on the errors or weaknesses of the previous models.
- Unlike bagging, which trains models independently, boosting builds a sequence of learners.

Working procedure

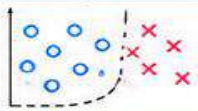
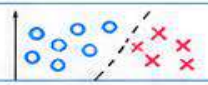
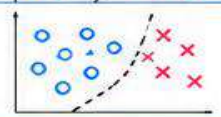
1. Train the first model on the training data.
2. Identify the errors or residuals made by the model.
3. Train the next model to focus on those errors or remaining patterns.
4. Repeat the process for several models.
5. Combine the predictions, often using a weighted combination.

How boosting works



Example

Suppose a model predicts whether students will pass or fail.

Model	Description	Result (illustration)
Model 1	Makes several incorrect predictions.	
Model 2	Focuses more on difficult cases (in AdaBoost) or fits remaining errors (in gradient boosting).	
Model 3	Continues the improvement process.	
Final prediction	Predictions from the models are combined to produce the final result.	

Types of boosting

Type	Description
AdaBoost (Adaptive Boosting)	• Increases the importance of observations that earlier models misclassified. • Trains a sequence of weak learners, giving higher weights to difficult cases.
Gradient Boosting	• Builds models to reduce the remaining errors by optimizing a loss function. • Each new model learns the residual errors of the previous model.
XGBoost (Extreme Gradient Boosting)	• An optimized gradient-boosting implementation. • Includes regularization and computational improvements. • Often provides high performance on structured datasets.

Advantages

1. Can provide high predictive accuracy.
2. Captures complex relationships in data.
3. Often performs well on structured datasets.
4. Can improve predictions by learning from earlier errors.

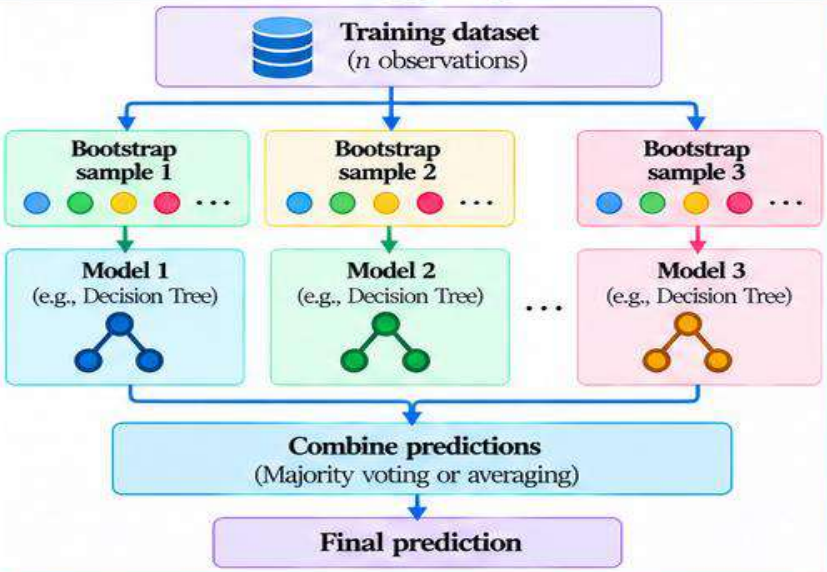
Limitations

1. Sequential training may take longer than bagging.
2. Requires careful hyperparameter tuning.
3. Can overfit if poorly configured.
4. Some boosting algorithms can be sensitive to noisy observations or incorrect labels.

Comparison of Bagging, Boosting and Random Forests

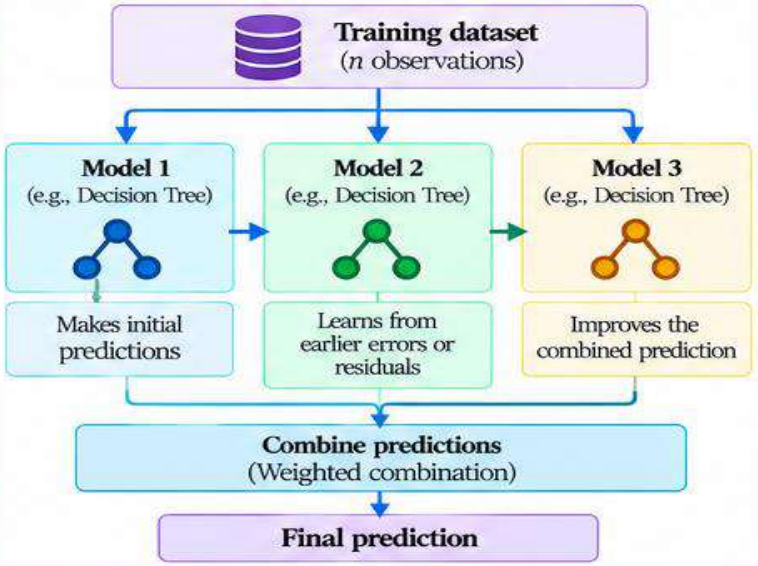
Feature	Bagging	Boosting	Random forests
Training method	Usually parallel	Sequential	Trees can be trained in parallel
Main idea	Train models on bootstrap samples	Improve predictions stage by stage	Combine randomized decision trees
Sampling	Usually bootstrap sampling	Depends on the algorithm	Usually bootstrap sampling
Feature randomness	Depends on base learner	Depends on the algorithm	Random feature subsets at each split
Main benefit	Reduces variance	Reduces predictive errors	Reduces tree variance and correlation
Example	Bagged decision trees	AdaBoost, XGBoost	Random Forest Classifier

Bagging



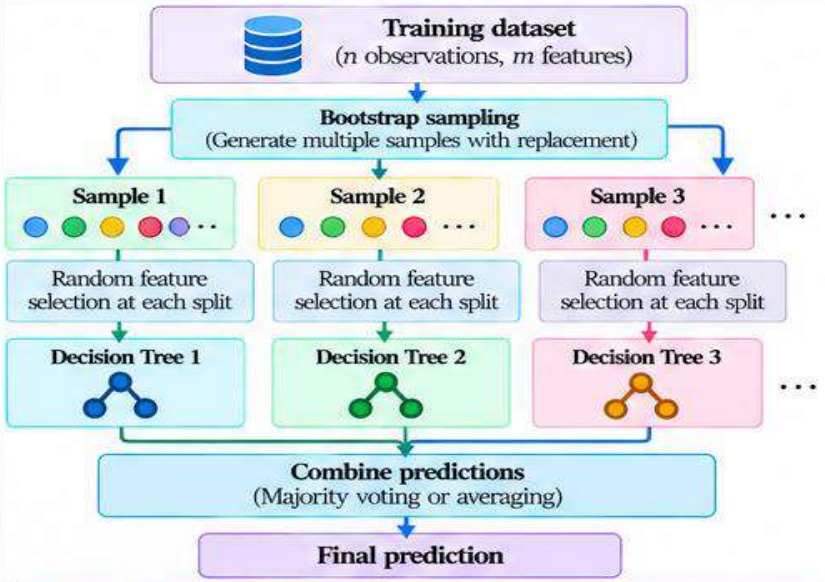
- Models are trained independently on different bootstrap samples.
- Predictions are combined using majority voting (classification) or averaging (regression).

Boosting



- Models are trained sequentially.
- Each new model focuses on the errors or weaknesses of the previous models.
- Predictions are combined, often using a weighted combination.

Random Forests



- Uses bootstrap samples (like bagging).
- Selects a random subset of features at each split (to reduce correlation between trees).
- Predictions are combined using majority voting or averaging.

Important relationship | A standard Random Forest combines bagging with random feature selection. Boosting is different because later models are trained to address errors or residuals from earlier stages.